ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

Please do not change the style of this document.

Question numbers 1–10 follow the same order as the notebook (top to bottom).

Student Information

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name: Yuan Hong, Lin**
- **Chinese Name: 林元鴻**
- **Email: andy021200@gapp.nthu.edu.tw**
- **Student ID: 114024504**
- **University: 國立清華大學**

Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

How to answer each question

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

Sources, honesty, and AI use

Acceptable vs. Unacceptable AI Usage: Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**
  - Copying assignment questions directly into an AI tool to generate answers.
  - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
  - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write "No" under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** Yes / No
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

Questions

Question 1: Data Filtering & Domain Relevance

Notebook: Section 3.2 — Question 1 (take home)

Task: Filter the dataframe **X** for the category **sci.med** and check its length.

Questions:

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

Answer:

1. There are 594 documents with category name equal to "sci.med"

2. When we look for patterns in data, different classes may have different patterns. If we mine all classes together, these class-specific patterns can be hidden, and generic patterns shared by every class tend to dominate the results. Therefore, before mining, we often need to look at specific classes separately (especially classes whose distribution is very different from the others), or even mine each class separately, so that we can find meaningful patterns for each class and compare them.

AI used for this answer: Yes

If yes, how: Translate and correct the grammar for question 1.2.

External references (links):

Question 2: Handling Missing Data

Notebook: Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

Task: In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

Question: If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

Answer:

Dropping 40% of the rows throws away a large amount of data, even though only the label is missing and the text itself is still complete. This reduces the information available for analysis. Also, labels are often not missing at random, so the remaining data can become a biased sample that distorts the class distribution and any patterns or models built on it.

A better alternative is self-training. First, train a classifier on the labelled data. Then use it to predict labels for the unlabelled data, and keep only the predictions with high confidence. This way, the text data can still be utilized.

AI used for this answer: Yes

If yes, how: Translate and correct the grammar, and asking for other strategies to deal with missing value problems(read the references by myself).

External references (links):

https://pandas.pydata.org/docs/user_guide/missing_data.html

https://scikit-learn.org/stable/modules/semi_supervised.html

Question 3: Data Sampling Risks

Notebook: Section 5.1 — Question 3 (take home)

Task: In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

Questions:

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

Answer:

1. No, the proportions are close but not exact.
2. Simple random sampling only preserves class proportions on average. On a highly imbalanced dataset the relative error is largest for the small classes. A minority class can be badly under-represented or missing entirely, so models and patterns built on the sample become biased toward the majority class.

AI used for this answer: Yes

If yes, how: Translate and correct the grammar for question 3.2

External references (links):

Question 4: Text Vectorization and Matrix Sparsity

Notebook: Section 5.3 — Question 4 (take home)

Task: In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

Questions:

1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

Answer:

1. The number of features increases, from 35482 to 35788.

2. Sparsity decreases, from 99.677% to 99.547% zeros. The 306 stop-word columns are only 0.9% of all columns, but they are dense. The number of non-zero cells rises from 258,503 to 365,886 (+41.5%), and the average number of distinct terms per document goes from 114.5 to 162.1. The matrix becomes less sparse and bigger, without adding any discriminative information.

AI used for this answer: Yes

If yes, how: Ask AI to write code for analyzing sparsity.

External references (links):

Question 5: Feature Selection — Variance Thresholding

Notebook: Section 5.3.1 — Question 5 (take home)

Task: In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

Questions:

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Answer:

Threshold	**Number of terms kept (keep == True)**
0.0	35482 (100%)
0.01	5089 (1403%)
0.05	1491 (4.2%)

1. The vocabulary shrinks sharply as the threshold rises. 0.01 already removes about 86% of the terms, and 0.05 removes about 96%.

2. Rare words are eliminated first, like the terms only appear once, typos, IDs and names.

3. A near-constant column takes almost the same value (0) in every documents, so it cannot separate classes or generalize. A word seen in a single document mostly adds noise and overfitting.

AI used for this answer: No

If yes, how:

External references (links):

Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)

Notebook: Section 5.3.2 — Question 6 (take home)

Task: Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

Questions:

1. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
2. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
3. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

Answer:

1. In Spearman filter, most of the top-correlated words are graphics terminology (like "graphics", "image", "3d"…). In Pearson filter, its mixes generic conversational words, header words and hardware terms.

2. Pearson measures the strength of a linear relationship between the raw values: cov(X,Y)/(σXσY). It is sensitive to magnitude and outliers. Spearman is Pearson computed on the ranks, so it measures any *monotonic* relationship and ignores how large the values are.

3. A word that appears in many in-class documents but with a heavy-tailed count.

AI used for this answer: Yes

If yes, how: explain the math formula of Pearson and Spearman correlation.

External references (links):

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.pearsonr.html

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.spearmanr.html

---

Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics

Notebook: Section 5.4.2 — Question 7 (take home)

Task: In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

Questions:

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?

2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

Answer (1) — Runtime and pattern count when lowering minSup:

Lowering minSup from 9 to 3 multiplies the number of maximal patterns by about 13×. For alt_atheism it goes from 710 to 9,547. Runtime grows by about 14×; for alt_atheism it goes from 0.06 s to 0.87 s. Many more itemsets pass the threshold, so the FP-tree has many more conditional branches to explore.

Answer (2) — Top-K k vs MaxFPGrowth minSup:

minSup is an absolute frequency cutoff, so you cannot know in advance how many patterns it will return, and a low value can explode. *k* fixes the size of the output to the *k* most frequent patterns, and the effective support threshold adapts to the data. For alt_atheism, k=100 implies a minimum support of 21 and k=500 a minimum support of 16. Top-K is rank-based; minSup is threshold-based.

Answer (3) — Maximal vs standard frequent patterns:

A maximal pattern is a frequent itemset with no frequent superset. Every subset of a frequent itemset is also frequent, so one maximal itemset of length L stands for 2^L − 1 frequent subsets without listing them.

AI used for this answer: Yes

If yes, how: Ask AI to fix the UnicodeEncodeError in the 113th code block, and ask the detail of the **Top-K FAE** and **MaxFPGrowth**.

External references (links): https://github.com/UdayLab/PAMI

Question 8: Dimensionality Reduction — Motivation

Notebook: Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

Task: After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

Questions:

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

Answer:

1. X_counts is a 2257 * 35,482 matrix, so each document is a point in a space with about 35k dimensions, one per word. A scatter plot has only 2 (or 3) axes. Picking any two word columns would show 2 out of 35k dimensions, and because 99.68% of the cells are zero, almost every document would land on (0, 0) on top of the others. High-dimensional distances also lose contrast (the curse of dimensionality).

2. Dimensionality reduction (PCA, t-SNE, UMAP) projects the documents into 2D or 3D first.

AI used for this answer: No

If yes, how:

External references (links):

Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

Notebook: Section 5.5 — Question 9 (take home) (after 2D/3D plots)

Task: In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

Questions:

1. Which method **(PCA or t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Answer:

1. t-SNE and UMAP show category-specific regions and islands. Both have a tight sci.med island; UMAP shows an alt.atheism lobe and a soc.religion.christian region. PCA collapses almost all documents into one heavily overlapping blob near the origin, with a few very long comp.graphics posts stretching the axes.

2. PCA is a linear projection that maximizes global variance. With raw counts, that variance is dominated by a few long documents and high-count terms, so the first two components describe outliers and document length rather than topics. Two linear directions also capture only a tiny share of the variance spread over 35k dimensions. t-SNE and UMAP are non-linear: they preserve *local* neighbourhoods, so documents that share vocabulary are placed together.

3. t-SNE optimizes the preservation of local neighbour probabilities, not global distances. The gaps between clusters, the cluster sizes and their densities carry no reliable meaning. They also change with the perplexity setting and the random seed (the notebook's t-SNE has no random_state). The plot should be read for local grouping only.

AI used for this answer: Yes

If yes, how: Translate and correct the grammar, and ask about the UMAP algorithm.

External references (links):

https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html

---

Question 10: One-Hot Binarization

Notebook: Section 5.6 — Question 10 (take home)

Task: In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

Questions:

1. How many binary columns are created to represent the 4 categories?
2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

Answer:

1. 4 binary columns. The binarized shape is (2257, 4), with classes 0–3 = alt.atheism, comp.graphics, sci.med, soc.religion.christian.

2. The raw codes make the algorithm assume the categories are ordered and equally spaced (ordinal/interval).

3. One-hot binarization gives each category its own orthogonal 0/1 dimension. There is no order, every pair of different categories is the same Euclidean distance apart (√2), and a linear model learns a separate weight for each category.

AI used for this answer: Yes

If yes, how: Translate and correct the grammar.

External references (links):

https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelBinarizer.html

---